ALLO INNOWARE — AI OPERATIONS MANAGER

Recurring Task Tracking, Pending Reports & Escalation Automation | Master Guide & Blueprint

Candidate: Sanath S Gowda
Role: AI Operations Manager
Prototype Link: docs.google.com/spreadsheets/d/1pt30OdGGjmo8_U7ogjAg3V4QRMNh3V6hm0Cz7OxmSdI
Target Domain: E-commerce & Operations Automation
Technology: Google Sheets, Apps Script, Webhooks, HTML Web App
Repository: github.com/Sanathgowda-17/recurring-ops-automation

1. THE REAL BUSINESS PROBLEM (THE 'WHY')

Allo Innoware is a rapidly scaling e-commerce brand operating across major marketplaces (Amazon, Flipkart) and D2C channels. As order volumes and operational touchpoints expand, recurring processes across inventory, vendor reconciliation, accounting closing, and daily team standups must execute flawlessly on fixed cadences without constant executive supervision.

The Core Failure Modes Before This Prototype:

- **Manual Cycle Drift:** If a 7-day task due on Tuesday was completed on Thursday, employees would manually recalculate dates incorrectly or forget the next due date entirely.
- **Duplicated Reporting Friction:** Team members had to check one sheet for pending items, perform work in an external portal, update a master tracker, and then log completion notes elsewhere.
- **The Daily Task Proof Vacuum:** Daily standup updates in WhatsApp groups produce no natural Ticket IDs. Without a structured proof protocol, managers had no lightweight way to verify completion.
- **Founder Chokehold:** Lacking automated multi-tiered escalation, operational delays were either completely overlooked or dragged straight to executive leadership, creating constant founder micro-management.

2. SYSTEM ARCHITECTURE & 6 CORE OPERATIONAL SHEETS

The system organizes operational state into 6 purpose-built sheets, separating data storage from employee interaction to prevent formula corruption and duplication:

Sheet Name	Primary Role	Operational Function
Config	System Control	Centralized parameters: Manager/Leadership emails, cutoffs (11:00 AM daily), SLA timers, Web App URL.
Task_Master	Catalog	Master definitions: Task IDs, names, assigned employees, cadences (Daily, 7d, 15d, 30d), and proof types.
Active_Tasks	Live Queue	The heartbeat engine tracking cycle instances (e.g. TSK-001-C01), due dates, status transitions, and audit timestamps.
Employee_Workbench	Single-Point UI	Filtered interface for employees. Enter Ticket ID/WhatsApp proof in Col H and submit with 1 click. Zero duplicate work.
Manager_Dashboard	Executive KPI	Overhead summary cards (Pending, Alerted, Escalated, Completed) and real-time employee compliance percentages.
Audit_Log	Immutable Ledger	Permanent event trail recording actor, timestamp, event type, and before/after status for all transactions.

3. BREAKDOWN OF THE 8 ASSIGNMENT TEST SCENARIOS

Each required test was engineered into the prototype and can be triggered via the custom Google Sheets menu:

• **Test 1 — Recurring Cycle Logic:** Completing a 7-day, 15-day, or 30-day task closes the active cycle and automatically spawns Cycle N+1 in Active_Tasks with the due date advanced exactly by the frequency offset. Daily tasks skip weekends to maintain working-day cadences.

• **Test 2 — Single-Point Completion:** Demonstrates that Employee C selects their name in Employee_Workbench, views only their pending tasks, types their Ticket ID (e.g. VEND-RECON-5510) directly into Column H, and clicks Submit. The master database updates without secondary manual entries.

• **Test 3 — Daily Task (WhatsApp Proof):** Deploys the practical 'Timestamp + Summary Snippet Protocol' (e.g., 10:45 AM - 'Ops Standup: 18 dispatches on schedule, zero delays'). Requires <15s of employee effort, provides verifiable proof against group chats, and auto-schedules the next business day.

• **Test 4 — Same-Day Manager Alert:** When a task passes cutoff + 30-min grace period, the escalation engine dispatches an HTML email to the manager on the same day containing task details, delay duration, and an embedded 1-Click Acknowledge button. Stamps Manager_Alert_Sent_At.

• **Test 5 — Manager Action & Acknowledgement:** Clicking the 1-Click button calls the deployed Apps Script Web App (doGet). The system marks status as 'Manager Acknowledged', records the manager timestamp, and places a hold on leadership escalation while follow-up occurs.

• **Test 6 — Leadership Escalation:** If 4 hours elapse post-alert without manager acknowledgement or resolution, the engine detects an SLA breach and fires an urgent high-priority HTML escalation email to Leadership (CCing Manager and Employee).

• **Test 7 — Timing SLA Matrix Logic:** Enforces distinct SLA tiers: Daily tasks trigger alert at 11:30 AM with a 4-hour manager window (escalating at 3:30 PM same day before COB). Periodic tasks alert next morning at 9:00 AM with a 24-hour resolution SLA.

• **Test 8 — Anti-Duplication & Idempotency:** Enforces LockService concurrency locks and atomic timestamp guards (Manager_Alert_Sent_At, Next_Cycle_Generated). Running the engine 3 times back-to-back verifies exactly 0 duplicate alerts and 0 duplicate cycles.

4. DAILY TASK TRACKING METHODOLOGY (TEST 3 IN-DEPTH)

The Challenge: Daily operational actions (such as posting morning dispatches in a WhatsApp group) do not natively produce portal ticket IDs. A simple yes/no checkbox invites dishonest ticking, while full API bots cost significant monthly tooling spend.

Our 2-Phase Operational Strategy:
1. **Immediate Protocol (Implemented):** The employee pastes `[Timestamp] - [Message Summary Snippet]` into Column H of the Workbench. This takes under 15 seconds, creates an audit log entry, and enables the manager to spot-check against the WhatsApp group in seconds.
2. **Enterprise Evolution Path:** As volume grows, a webhook listener (Twilio/WhatsApp Cloud API) can intercept `#OPS_STANDUP_DONE` hashtags in the group and post directly to the Apps Script doGet/doPost endpoint, achieving zero-touch automation.

5. GOLDEN INTERVIEW PREPARATION (EXACT FOUNDER QUESTIONS & ANSWERS)

Q1: Why did you build an Employee Workbench instead of letting team members edit Active_Tasks directly?

Answer: Direct master editing creates human error and duplication. When 4 employees edit a 20-column table simultaneously, formulas get broken and rows overwritten. The Workbench acts like a clean front-end filter—employees see only their tasks, enter proof in a dedicated green cell, click Submit, and the system handles the database and cycle generation in the background with zero duplicate work.

Q2: How does the system guarantee zero duplicate alert emails or cycle generations?

Answer: Through two enterprise-grade safeguards: First, script concurrency locking using Google Apps Script's LockService.getScriptLock(), which queues requests and prevents race conditions. Second, atomic timestamp guards (Manager_Alert_Sent_At, Leadership_Escalation_Sent_At, Next_Cycle_Generated). Once a flag is stamped, subsequent scans immediately ignore the record.

Q3: How does your architecture specifically reduce founder dependency?

Answer: Before this system, any delay caused the employee to go to the founder or the founder to hunt for updates. Our architecture introduces an automated 2-tier SLA shield: the system alerts the manager first with a 1-click email action button. The founder is only notified if the manager neglects their 4-hour SLA or if the task remains unresolved beyond critical thresholds. Routine issues stay at the operational tier.

Q4: What happens if a daily task falls on a Friday or weekend?

Answer: The recurrence engine incorporates working-day intelligence. When a daily task is completed on Friday, the calculateNextDueDate function automatically skips Saturday and Sunday, setting the next cycle due date for Monday morning at 11:00 AM, maintaining realistic operational cadences.